



# SACHIN PATIL

CYBER SECURITY

## VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Target: sachincybersecurity.page.gd

Portfolio — Web Application Security Assessment

### INTRODUCTION:

This report presents the results of a penetration test conducted against the Portfolio web application hosted at sachincybersecurity.page.gd. The objective of this engagement was to identify security vulnerabilities within the application's authentication mechanisms, session handling, input validation, and overall configuration, and to provide actionable remediation guidance for each issue identified.

A total of 9 findings were identified during this assessment, ranging in severity from Critical to Low. Findings in this report are listed in order of severity, starting with the most critical issue. Each finding includes the finding name, severity rating, affected location, a description of the issue, proof-of-concept evidence, and recommended remediation steps.

## EXECUTIVE SUMMARY

A penetration test was conducted against the Portfolio Admin web application hosted at [sachincybersecurity.page.gd](http://sachincybersecurity.page.gd). The objective of the engagement was to identify security weaknesses in the application's authentication mechanisms, session management, input handling, and overall configuration, and to provide clear, actionable remediation guidance for each issue identified.

During testing, a total of 9 distinct vulnerabilities were identified, spanning Critical, High, Medium, and Low severity. The most significant issue is a Critical SQL Injection vulnerability in the admin login form, which allows complete authentication bypass without valid credentials. This finding is compounded by the absence of rate limiting on the same login endpoint, which would otherwise have slowed or blocked automated exploitation attempts. Taken together, these two issues represent the most immediate risk to the application and are recommended for remediation as a first priority.

The remaining findings relate to weaknesses in defense-in-depth controls, including a reflected Cross-Site Scripting (XSS) vulnerability, absence of multi-factor authentication on the admin panel, insecure session cookie attributes, verbose error and stack trace disclosure, cleartext transmission of credentials at the application layer, missing HTTP security headers, and autocomplete left enabled on sensitive login fields. While individually lower in severity, several of these issues would meaningfully increase the impact of the Critical and High findings if left unaddressed — for example, insecure cookie attributes increase the impact of the XSS finding, and verbose error disclosure assists further exploitation of the SQL injection vulnerability.

### Summary of Findings by Severity:

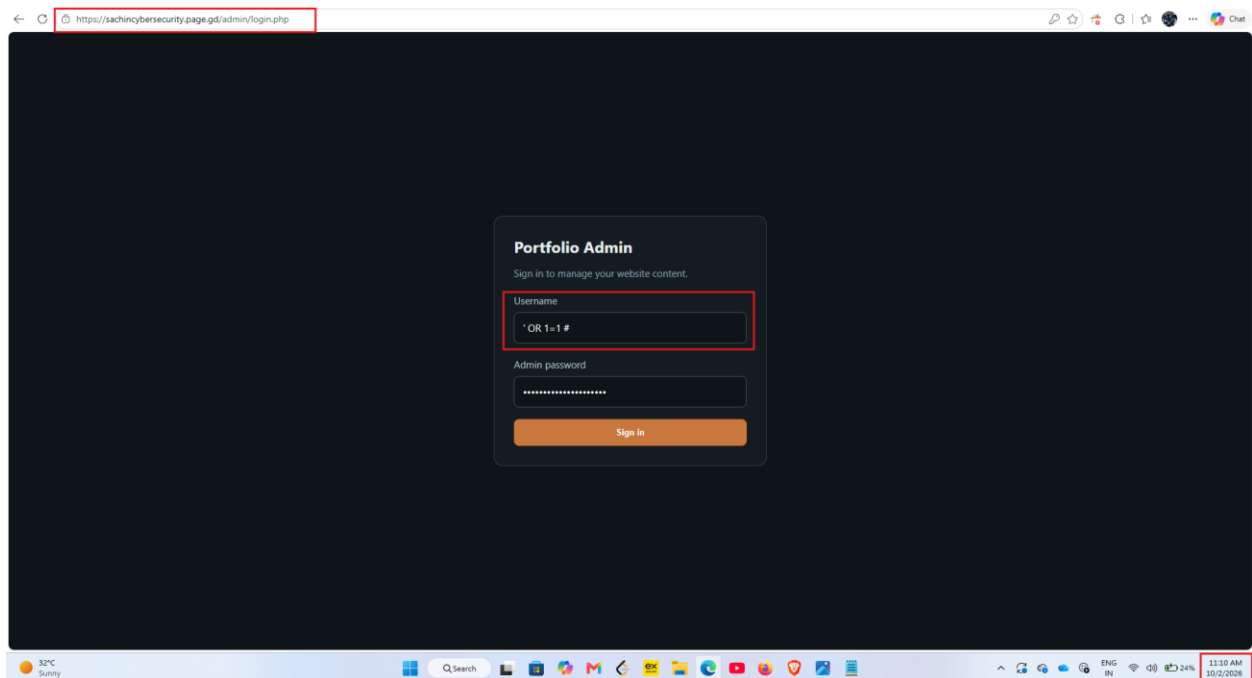
Critical: 2 findings High: 1 finding Medium: 4 findings Low: 2 findings

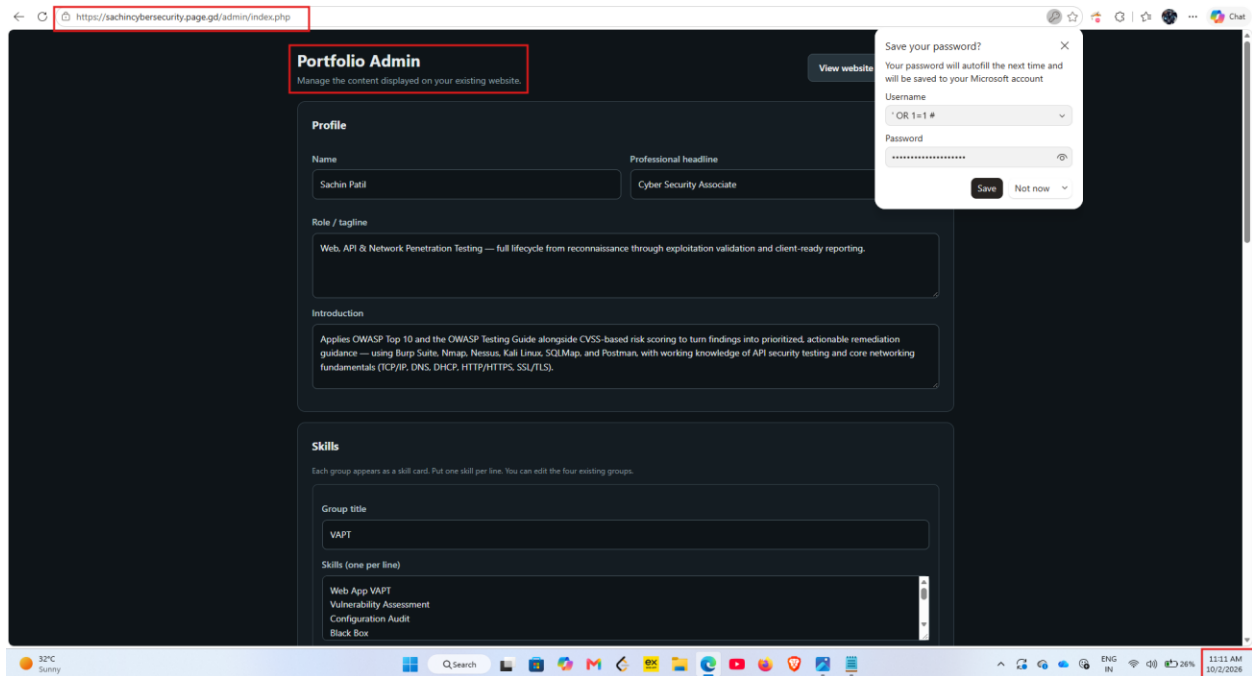
It is recommended that the Critical and High severity findings be remediated immediately, followed by the Medium severity findings as part of the next release cycle, and the Low severity findings addressed as general security hardening. Detailed descriptions, proof-of-concept evidence, and remediation steps for each finding are provided in the sections that follow, ordered by severity from Critical to Low.

| Sr. No. | Vulnerability / Finding                                                                            | Severity |
|---------|----------------------------------------------------------------------------------------------------|----------|
| 1       | SQL Injection in Admin Login Form (Authentication Bypass)                                          | Critical |
| 2       | Reflected Cross-Site Scripting (XSS)                                                               | Critical |
| 3       | Missing Rate Limiting on Login Endpoint                                                            | High     |
| 4       | Transmission of Credentials Without Additional Protection (Cleartext Username/Password in Request) | Medium   |
| 5       | Missing Two-Factor Authentication (2FA) on Admin Login                                             | Medium   |
| 6       | Insecure Session Cookie Attributes                                                                 | Medium   |
| 7       | Verbose Error Message and Stack Trace Disclosure                                                   | Medium   |
| 8       | Missing Security Headers                                                                           | Low      |
| 9       | Autocomplete Not Disabled on Sensitive Login Fields                                                | Low      |

**FINDING NAME: SQL Injection in Admin Login Form (Authentication Bypass)****SEVERITY: Critical****LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>**DESCRIPTION:**

The admin login form at the endpoint above does not sanitize or parameterize user-supplied input before using it in a backend SQL query. Submitting the payload ' OR 1=1 # in the Username field, combined with any password value, altered the logic of the underlying SQL query and allowed authentication to be bypassed without valid credentials. After submission, the application granted access to the "Portfolio Admin" dashboard, confirming that the injected input was executed directly against the database

**POC:**

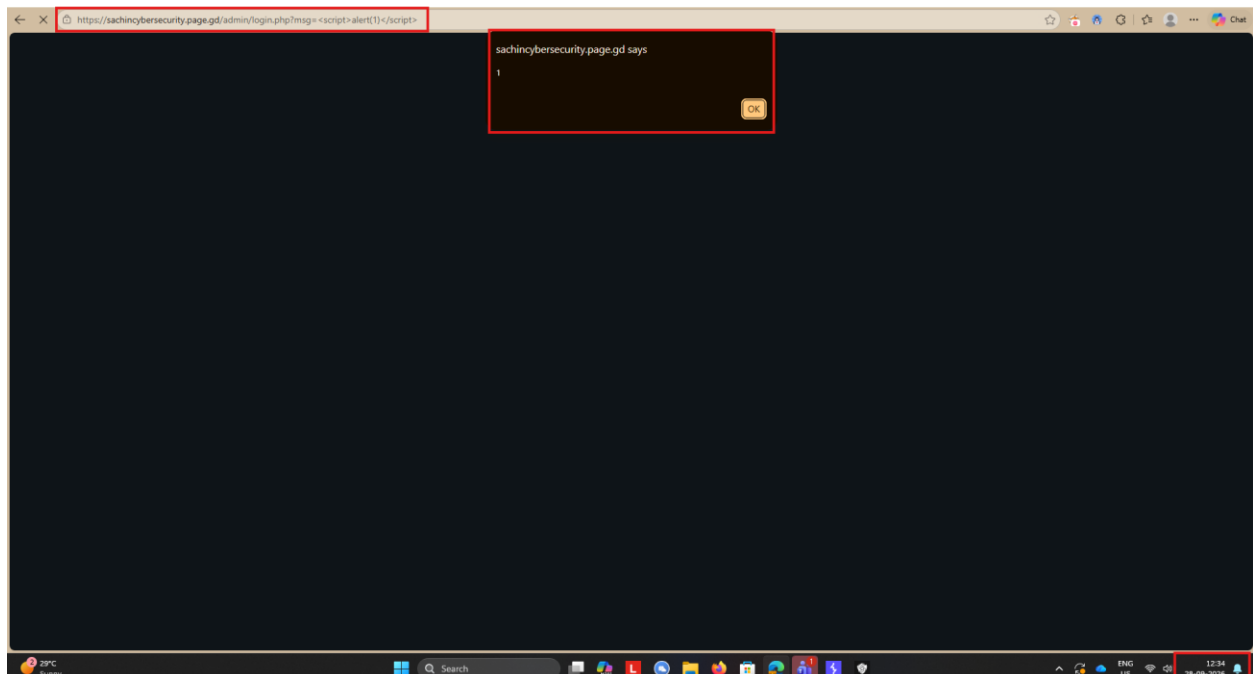


## REMEDIATION:

Rewrite all database queries to use parameterized statements / prepared statements (e.g., PDO or mysqli with bound parameters) instead of concatenating user input directly into SQL strings. Apply strict server-side input validation and allow-listing on the login fields. Enforce the principle of least privilege for the database account used by the application. Deploy a Web Application Firewall (WAF) as a compensating control, and conduct a full code review of all other input points for similar injection flaws

**FINDING NAME: Reflected Cross-Site Scripting (XSS)****SEVERITY: Critical****LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>**DESCRIPTION:**

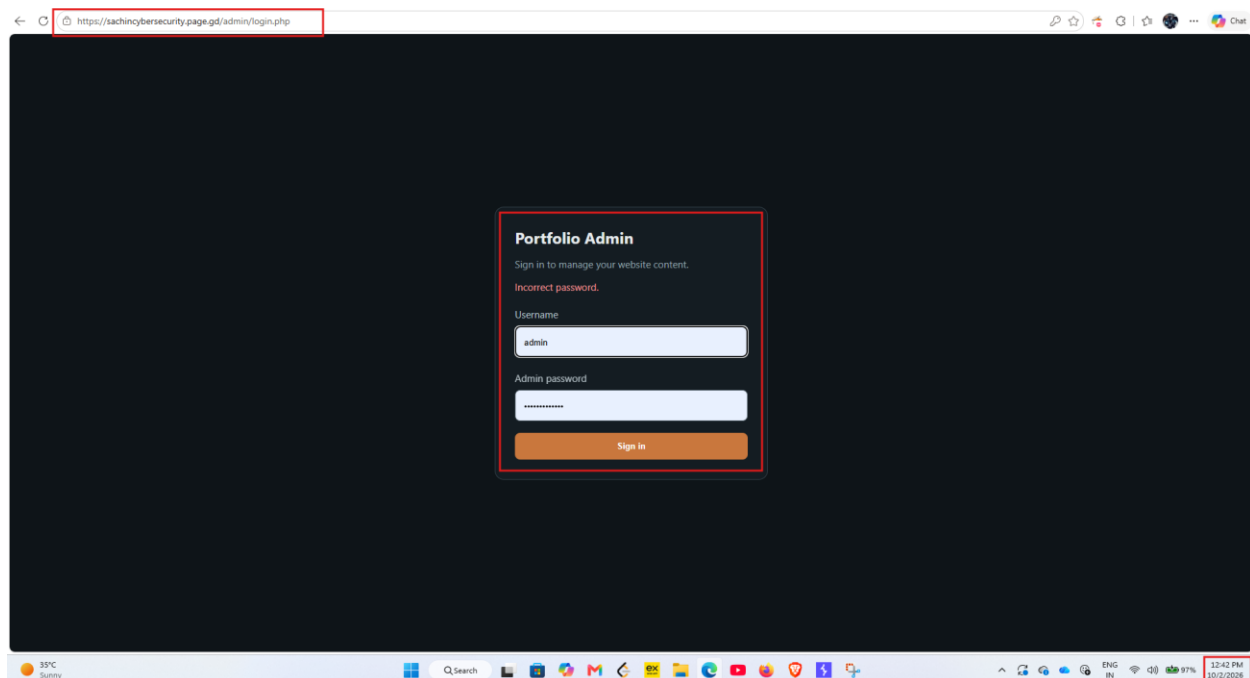
The 'msg' query parameter on the login page is reflected back into the HTML response without output encoding or sanitization. Injecting a <script> payload into this parameter caused the script to execute in the victim's browser, as demonstrated by the triggered JavaScript alert box. Because this occurs on an authentication page, an attacker could craft a malicious link and use it for session hijacking, credential phishing, or redirecting victims to an attacker-controlled page

**POC:****REMEDIATION:**

Encode all user-controllable output using context-appropriate encoding (HTML entity encoding for HTML body context) before rendering it in the page. Avoid directly reflecting query parameters into the DOM. Implement a strict Content-Security-Policy (CSP) header to reduce the impact of any residual XSS. Validate and sanitize all input server-side, and use templating engines that auto-escape output by default

**FINDING NAME: Missing Rate Limiting on Login Endpoint****SEVERITY: High****LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>**DESCRIPTION:**

Repeated login attempts with an incorrect password against the admin login endpoint were not throttled, locked out, or challenged with a CAPTCHA. The application simply returned an "Incorrect password" message and allowed further immediate attempts. This absence of rate limiting or account lockout makes the login form susceptible to automated brute-force and credential-stuffing attacks against the admin account

**POC:****REMEDIATION:**

Implement account lockout or exponential back-off after a defined number of failed login attempts, IP- and account-based rate limiting, and a CAPTCHA/challenge after repeated failures. Log and alert on repeated failed authentication attempts to support detection of brute-force activity

**FINDING NAME:** Transmission of Credentials Without Additional Protection (Cleartext Username/Password in Request)

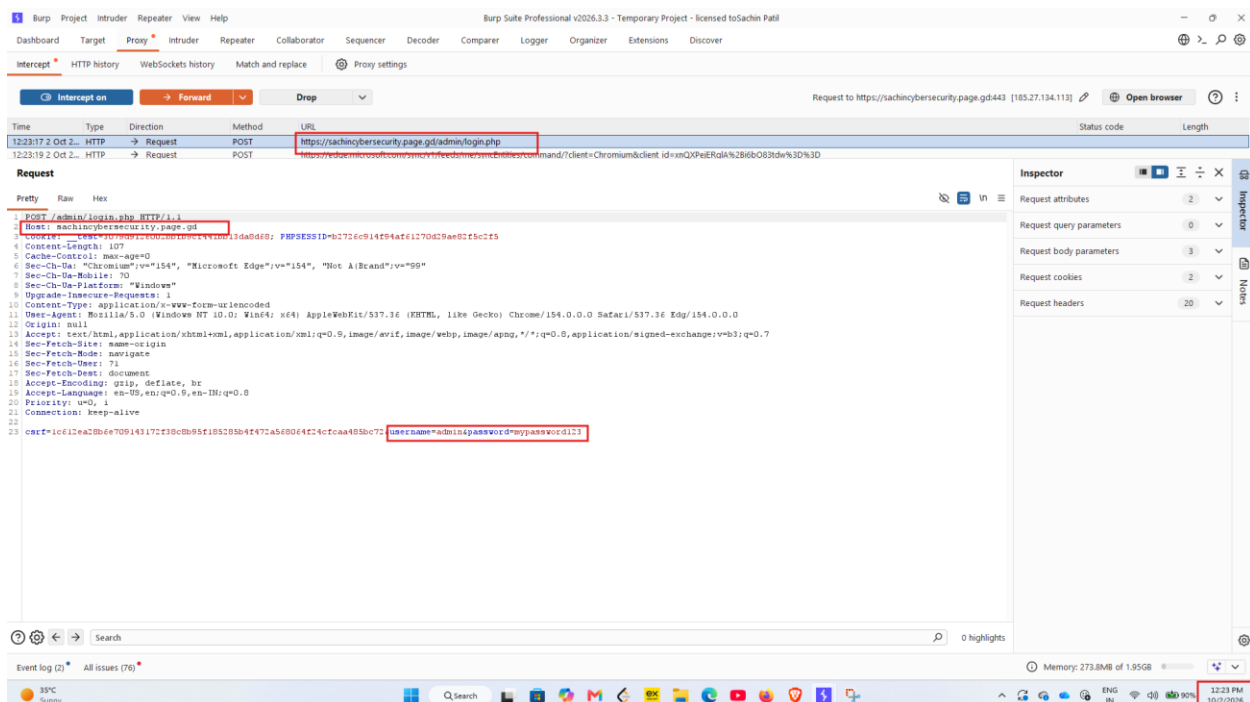
**SEVERITY:** Medium

**LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>

#### DESCRIPTION:

Intercepting the login request with Burp Suite shows that the username and password are submitted as plain, unencrypted form parameters (username=admin&password=mypassword123) in the POST body. While the connection itself uses HTTPS, the credentials are not hashed, salted, or otherwise protected at the application layer before transmission, and the CSRF token and session cookie are also visible in the same request. Any point of TLS interception (misconfigured proxy, compromised endpoint, malicious browser extension, or downgrade attack) would expose the credentials in cleartext

#### POC:

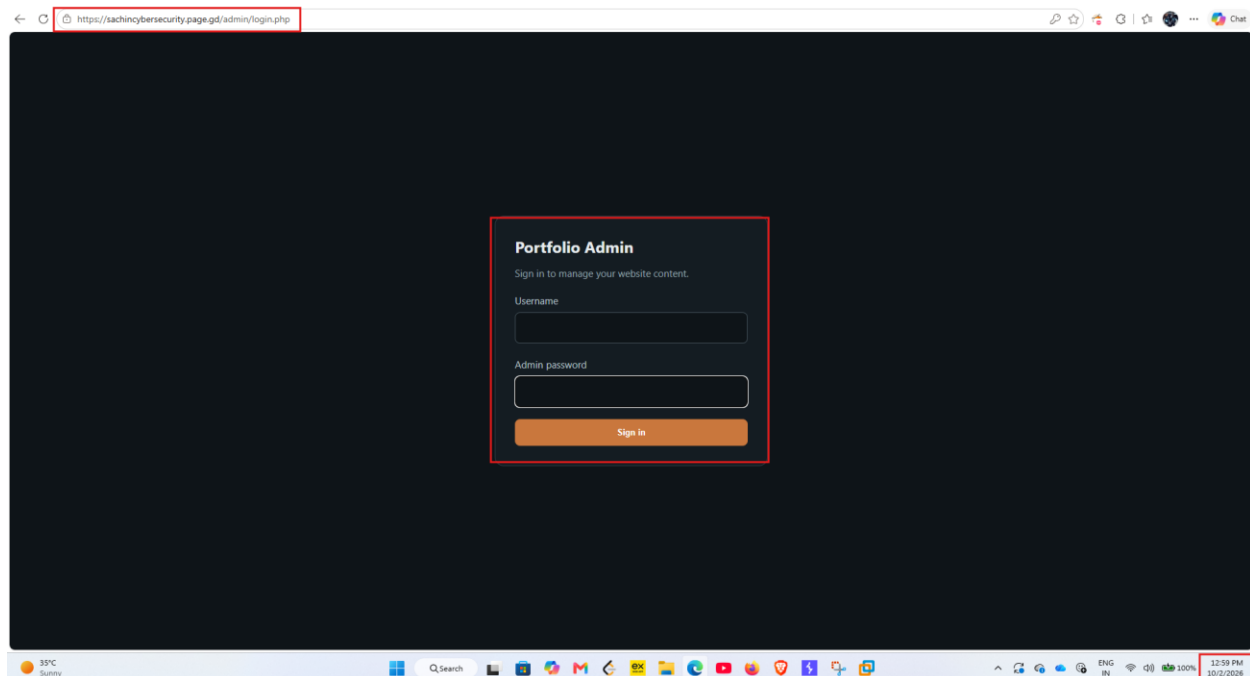


#### REMEDIATION:

Ensure HSTS (HTTP Strict-Transport-Security) is enforced to prevent TLS downgrade. Consider additional application-layer protections such as client-side hashing combined with server-side salted hashing, and short-lived, single-use CSRF tokens. Avoid logging raw credential values anywhere in server logs, and enforce secure cookie flags on the associated session

**FINDING NAME: Missing Two-Factor Authentication (2FA) on Admin Login****SEVERITY: Medium****LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>**DESCRIPTION:**

The administrative login page only requires a username and password, with no second authentication factor (OTP, authenticator app, email/SMS verification, etc.). Given that this endpoint grants access to a privileged administrative panel, relying solely on single-factor authentication significantly increases the impact of credential theft, brute forcing, or the authentication-bypass issue documented above

**POC:****REMEDIATION:**

Implement multi-factor authentication (TOTP-based authenticator app, WebAuthn/security key, or at minimum email/SMS OTP) for all administrative accounts before granting access to the admin panel. Enforce 2FA enrollment at account creation and disallow administrative actions until a second factor is configured

## FINDING NAME: Insecure Session Cookie Attributes

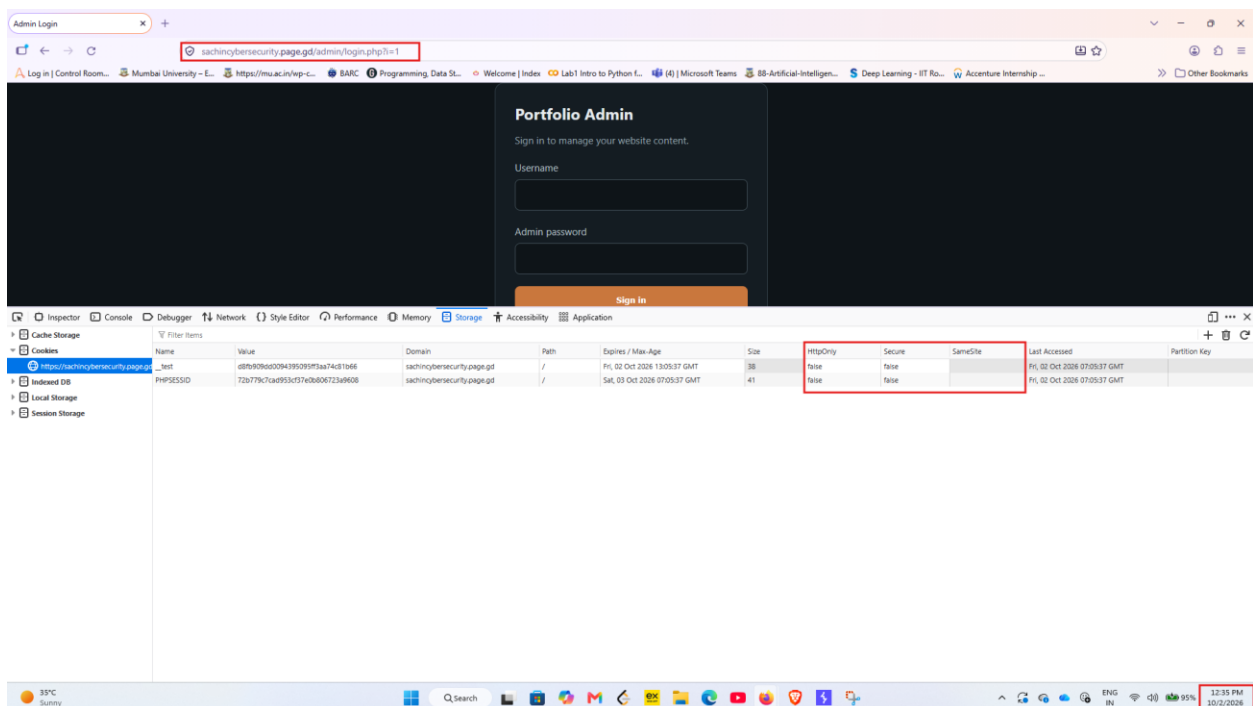
**SEVERITY:** Medium

**LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>

### DESCRIPTION:

Inspecting the browser's cookie storage for the application shows that the session cookie (PHPSESSID) and a secondary cookie (\_test) are both set with HttpOnly, Secure, and SameSite left unset/false. Without HttpOnly, cookies are accessible to client-side JavaScript, increasing the impact of any XSS vulnerability (such as the one identified above) by allowing session theft. Without Secure, cookies may be transmitted over unencrypted channels, and without SameSite, the cookies are more exposed to CSRF-style attacks

### POC:

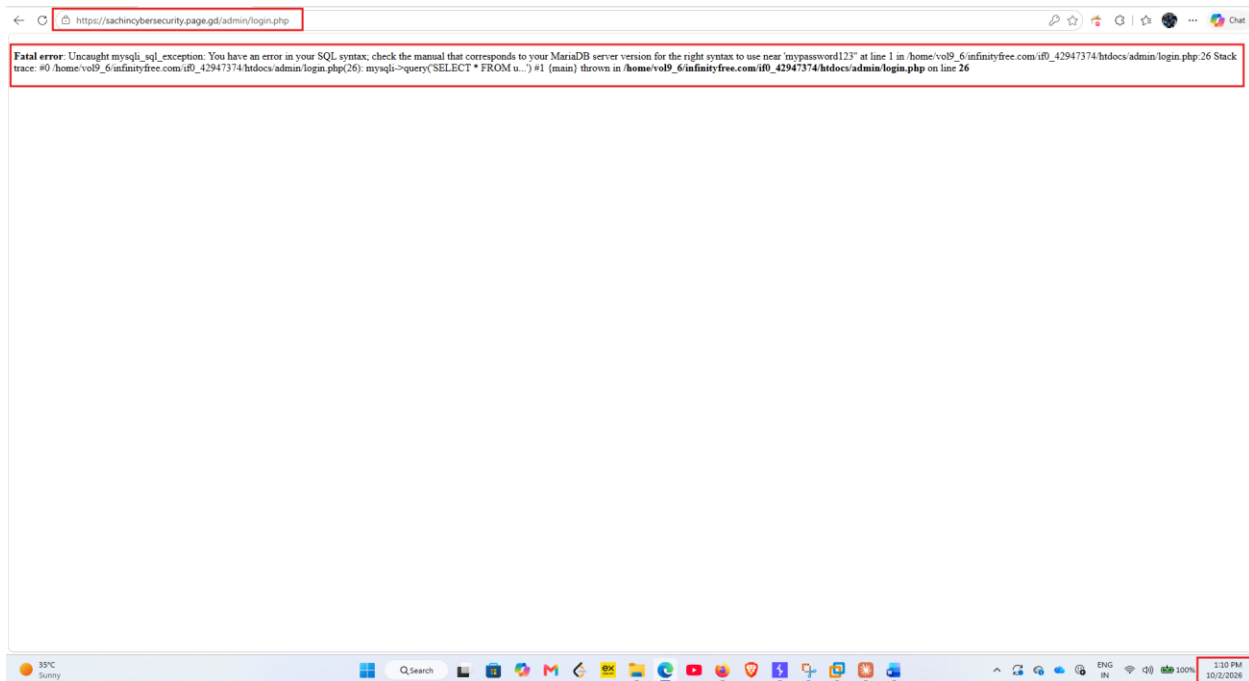


### REMEDIATION:

Set the HttpOnly flag on all session cookies to prevent JavaScript access. Set the Secure flag to ensure cookies are only sent over HTTPS. Set SameSite=Strict or Lax to mitigate CSRF. In PHP, configure these via `session.cookie_httponly`, `session.cookie_secure`, and `session.cookie_samesite` in `php.ini`, or explicitly via `session_set_cookie_params()` before `session_start()`

**FINDING NAME: Verbose Error Message and Stack Trace Disclosure****SEVERITY: Medium****LOCATION:** <https://sachincybersecurity.page.gd/admin/login.php>**DESCRIPTION:**

Triggering a malformed database query via the login form (as part of SQL injection testing) caused the application to return a raw PHP/MySQLi fatal error and full stack trace directly to the browser. The disclosed error reveals the underlying database driver (mysqli), the full server-side file system path (/home/vol9\_6/infinityfree.com/inf0\_42947374/htdocs/admin/login.php), the exact line number of the vulnerable query, and a fragment of the SQL statement structure. This information significantly assists an attacker in further exploiting the SQL injection vulnerability and understanding the server environment

**POC:****REMEDIATION:**

Disable `display_errors` and verbose error reporting in the production PHP configuration; log detailed errors server-side only. Return generic, user-facing error messages for all authentication failures. Wrap all database calls in proper exception handling so that unexpected query failures never surface raw driver output to the client

**FINDING NAME:** Missing Security Headers**SEVERITY:** Low**LOCATION:** <https://sachincybersecurity.page.gd/>**DESCRIPTION:**

An automated header-analysis scan of the target site found that 9 recommended security headers are absent: X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Content-Security-Policy, Referrer-Policy, Permissions-Policy, Cross-Origin-Embedder-Policy, Cross-Origin-Resource-Policy, and Cross-Origin-Opener-Policy. The absence of these headers increases exposure to clickjacking, MIME-sniffing attacks, protocol downgrade, and cross-origin data leakage

**POC:**

```
(kali㉿kali)-[~/shcheck]
$ ./shcheck.py https://sachincybersecurity.page.gd/

> shcheck.py - santoru .....

Simple tool to check security headers on a webserver

[*] Analyzing headers of https://sachincybersecurity.page.gd/
[*] Effective URL: https://sachincybersecurity.page.gd/
[!] Security header missing: X-Frame-Options
[!] Security header missing: X-Content-Type-Options
[!] Security header missing: Strict-Transport-Security
[!] Security header missing: Content-Security-Policy
[!] Security header missing: Referrer-Policy
[!] Security header missing: Permissions-Policy
[!] Security header missing: Cross-Origin-Embedder-Policy
[!] Security header missing: Cross-Origin-Resource-Policy
[!] Security header missing: Cross-Origin-Opener-Policy

[!] Analyzing headers for https://sachincybersecurity.page.gd/
[+] 0 security header(s) present
[-] 9 security header(s) missing
```

**REMEDIATION:**

Configure the web server/application to send the missing headers: Strict-Transport-Security (with an appropriate max-age and includeSubDomains), X-Content-Type-Options: nosniff, X-Frame-Options: DENY or SAMEORIGIN (and/or frame-ancestors in CSP), a restrictive Content-Security-Policy, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy limiting unneeded browser features, and the Cross-Origin-\* isolation headers where applicable

## FINDING NAME: Autocomplete Not Disabled on Sensitive Login Fields

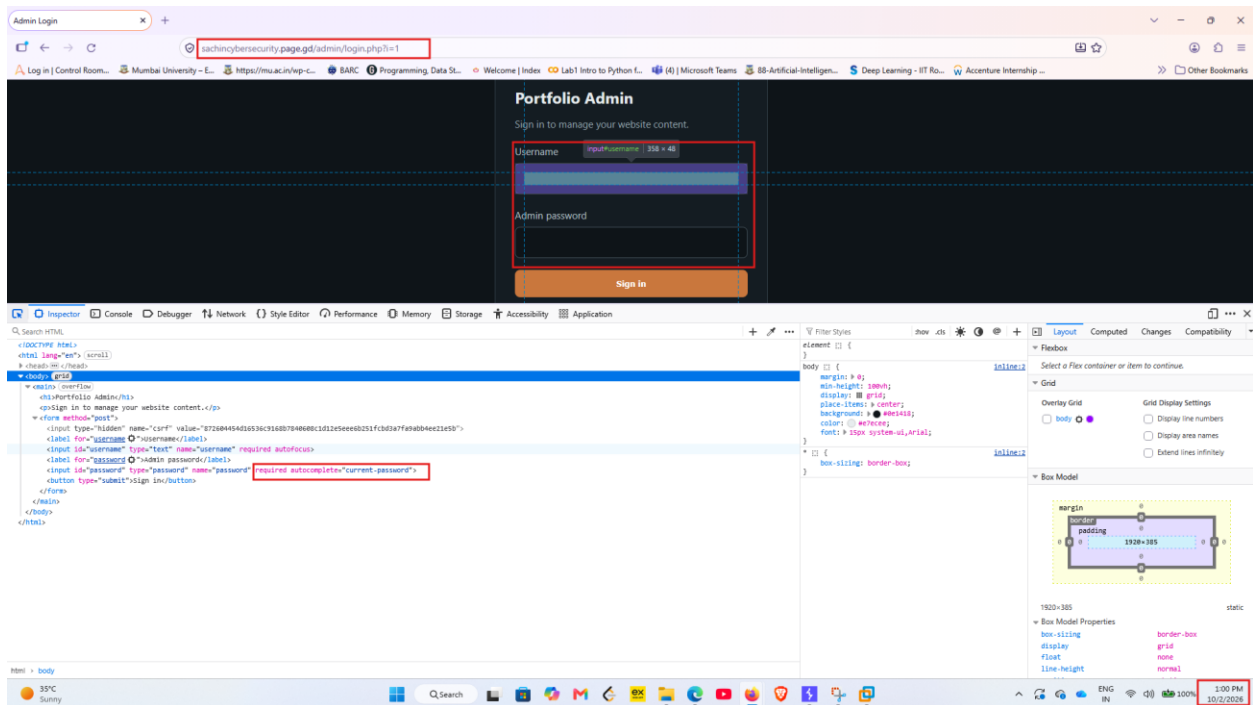
SEVERITY: **Low**

LOCATION: <https://sachincybersecurity.page.gd/admin/login.php>

### DESCRIPTION:

Inspection of the login form's HTML shows the password field is rendered with `autocomplete="current-password"` (and the username field has no autocomplete restriction either), rather than disabling autocomplete for this sensitive administrative login form. On a shared or public computer, browsers may offer to store and auto-fill these admin credentials, increasing the risk of unauthorized access if the device is later used by another person

### POC:



### REMEDIATION:

Set `autocomplete="off"` (or `"new-password"` where appropriate) on the username and password fields of the administrative login form. Pair this with organizational guidance against using autofill/password-saving on shared or public machines for privileged accounts